

Data Communication – Project

An application must be built that enables real-time messaging (chat) between a group of participants on the same LAN (must be registered first).

1. Plan the application that includes all the work steps

- a. Setting up the server
- b. Setting up the client
- c. Registering to the system
- d. Registering to the group
- e. Disconnecting from the system
- f. Disconnecting from the group
- g. Sending a message
- h. Receiving a message
- h. Creating a group

2. It is recommended to add –

- a. Planning and implementing encryption of the information that passes through the chat
- b. Implementation with an improved UI

3. All management operations between the clients and the server will be performed under the TCP protocol

4. All message transfers in the chat will be performed in multicast under the UDP protocol
5. Complete the theoretical material that you lack in order to implement the application
6. Precisely define the message protocol between the various components in the network
7. Perform detailed planning
8. Implement the application
9. You can use AI to generate code.

However, you must use the library of data structures we built based on the API in the H file you received. You must also adhere to the design we defined.

Project UI –

When the client loads, the following screen will be displayed -

1. registration
2. login
3. exit

When clicking on option 1 or 2 (registration or login), a request for a user name and password will be displayed.

The user name and password will be sent to the server for confirmation according to the selected option -

1. registration - The user name must be unique. If it already exists, an error will be returned
2. login - The user name must already be defined on the server and the password must be correct. If the conditions are not met, an error will be returned. If the login is correct, the client will move to the second screen that will allow creation and work with groups to chat.
3. exit - Orderly closing of the client. Closing the channel with the server, releasing all memory allocations and exiting the program.

Screen 2 -

When moving to screen 2, the following options will be displayed -

1. create group
2. join group
3. leave group
4. logout

When clicking on -

1. create group - the user will have to tap on a group name. The group name will be sent to the server for confirmation. If the name chosen is new and does not appear on the server and the server was able to open a new group with it, the server will send a response that the operation was successful and will automatically add the client to the group. When sending the response to the client, it will send data about the group as specified in the group joining process. If

the group opening failed, a response will be returned stating that the operation failed.

2. join group - the user will have to tap on the group name. The group name will be sent to the server. If the group does not exist, an error will be returned. If the group exists, the server will send the multicast address and port for the group to the client.

The client will open 2 chat windows for this group - one window for writing messages and sending them to the group and a second window for displaying all messages in the group.

3. leave group - the user will have to type the group name. The group name will be sent to the server. If the group does not exist, an error will be returned. If the group exists, the server will remove the client from the group and return OK to the client. The client will close the chat windows of that group and perform the other required actions such as releasing memory allocations if necessary, etc.

4. logout - the message will be sent to the server. The server will remove the client from all groups to which it is connected, will switch the client to an inactive state and return OK to the client. The client will close all windows of the groups to which it is connected, will clear all allocations, etc. and will switch to displaying the first screen.

When does a group close?

A group with no clients connected to it will close automatically. The server will manage the number of clients connected to each group. Every time a client joins

the group, the counter will be incremented by one. Every time a client leaves the group, the counter will be decremented by one. If the counter reaches zero, the group will close.

How can I open windows for chat?

The windows can be opened using the function - system.

In the function parameter, you can use gnome-terminal to open a terminal and tell it to run our program - you need to learn the subject and it is worth using AI.

How can I close a window from within our program?

Of course, you can use the kill function that we learned. The problem is that the system function that we used does not return the PID of the process that we opened. Therefore, the program that is running in the window that we open needs to find out its PID using the getpid function and send the PID to the client using the message queue that we learned.

How to build application that send or receive in multicast on LAN –

You can find a lot of documentation online including examples of C code.

Search for - multicast on lan

Note that you need to develop 2 small applications.

One that waits to receive a message from the user and sends the message to the multicast address. The other waits to receive a message from the multicast address and prints the message to the screen.